Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Основи розробки програмного забезпечення на платформі Java»
Комп'ютерний практикум №2.
Відношення між класами

Виконав: гр. ІП-з41 Черепов Олександр Павлович

Перевірив: старший викладач Дифучина Олександра Юріївна

Оцінка: _____

Дата оцінки: _____

2026

Комп’ютерний практикум №2

Тема роботи: Відношення між класами.
Мета роботи: Дослідити механізм спадкування класу.

Завдання до роботи

1. Створити класи у відповідності до варіанту завдання, використовуючи класи, побудовані на попередній лабораторній роботі. У головній програмі створити масив екземплярів батьківського класу та виконати розрахункові дії з використанням методів класу. Дослідити відношення спадкування, агрегування та залежності класів: розробка екземплярів класу-агрегата з різними параметрами, виклик методів для екземплярів класу, що агрегуються, розробка класів-нащадків, виклик перевизначених методів для екземплярів класів-нащадків і т.п. Запропонувати свої класи, які розширюють та доповнюють функціональні можливості Вашого пакету класів, та розробити їх. **50 балів**
2. Запропонувати в одному з класів використання вкладеного класу та локального класу. Дослідити доступ до елементів вкладеного класу з його зовнішнього класу та з інших класів. Дослідити доступ до елементів локального класу з зовнішнього класу. **30 балів.**
3. Опрацювати створення масивів екземплярів об’єктів мовою Java та основні прийоми обробки масивів: відшукання заданого елемента, відшукання найбільшого(найменшого) значення, сортування масиву, копіювання масиву. **20 балів**.

Варіант: 1.6) Клас «співробітник» - клас «аудиторія» - клас «лаборант»; 2.6) Розробити метод, який знаходить усі аудиторії, за які відповідає заданий співробітник.

Реалізація

Структура файлів:

```
programme/
├── auditory/
│   └── Auditory.java
├── employee/
│   ├── Employee.java
│   └── Laborant.java
└── Main.java
```

Лістинг програми:

programme /auditory/Auditory.java:

```
package auditory;

public class Auditory {
    private static int nextId = 1;

    private int id;
    private int floor;
    private String forSubject;
    private int capacity;

    public class Equipment {
        private String name;
        private boolean active;

        public Equipment(String name, boolean active) {
            this.name = name;
            this.active = active;
        }

        public String getStatus() {
            return name + " is " + (active ? "active" : "inactive") + " in
auditory " + id;
        }
    }

    public Auditory() {
        id = nextId++;
        this.floor = -1;
        this.forSubject = "Unknown";
        this.capacity = 0;
    }

    public Auditory(int floor, String forSubject, int capacity) {
        id = nextId++;
        this.floor = floor;
        this.forSubject = forSubject;
        this.capacity = capacity;
    }

    public int getId() {
        return id;
    }

    public int getFloor() {
        return floor;
    }
```

```
    public void setFloor(int floor) {
        if (floor < 0) {
            throw new IllegalArgumentException("Floor cannot be negative.");
        }
        this.floor = floor;
    }

    public String getForSubject() {
        return forSubject;
    }

    public void setForSubject(String forSubject) {
        if (forSubject == null || forSubject.trim().isEmpty()) {
            throw new IllegalArgumentException("Subject cannot be null or
empty.");
        } else if (forSubject.length() > 100 || forSubject.length() < 2) {
            throw new IllegalArgumentException("Subject must be between 2 and
100 characters.");
        }
        this.forSubject = forSubject;
    }

    public int getCapacity() {
        return capacity;
    }

    public void setCapacity(int capacity) {
        if (capacity < 0) {
            throw new IllegalArgumentException("Capacity cannot be
negative.");
        }
        this.capacity = capacity;
    }

}
```

programme/employee/Employee.java:

```
package employee;
import auditory.Auditory;

public class Employee {
    private static int totalEmployees = 0;
    private static int nextId = 1;

    private int id;
    private String name;
    private double salary;
    private Auditory[] auditories;

    public Employee() {
```

```
        id = nextId++;
        this.name = "Unknown";
        this.salary = 0;
        this.auditories = null;
        totalEmployees++;
    }

    public Employee(String name, double salary, Auditory[] auditories) {
        id = nextId++;
        this.name = name;
        this.salary = salary;
        this.auditories = auditories;
        totalEmployees++;
    }

    public int getId() {
        return id;
    }

    public String getEmployeeData() {
        return "ID: " + id + ", Name: " + name + ", Salary: $" + salary;
    }

    public static int getTotalEmployees() {
        return totalEmployees;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalArgumentException("Name cannot be null or
empty.");
        } else if (name.length() > 100 || name.length() < 2) {
            throw new IllegalArgumentException("Name must be between 2 and
100 characters.");
        }

        this.name = name;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        if (salary < 0) {
            throw new IllegalArgumentException("Salary cannot be negative.");
        }
```

```
        this.salary = salary;
    }

    public Auditory[] getAuditories() {
        return auditories;
    }

    public void setAuditories(Auditory[] auditories) {
        int n = auditories.length;
        int[] auditoryIds = new int[n];

        for (int i = 0; i < n; i++) {
            auditoryIds[i] = auditories[i].getId();
        }

        for (int i = 0; i < n; i++) {
            for (int j = i+1; j < n; j++) {
                if(auditories[i].getId() == auditories[j].getId()) {
                    throw new IllegalArgumentException("Cannot set
auditories: Duplicate auditories found.");
                }
            }
        }

        this.auditories = auditories;
    }

    public void clearAuditories() {
        auditories = null;
    }

    public void addAuditory(Auditory auditory) {
        if (auditories == null) {
            auditories = new Auditory[1];
            auditories[0] = auditory;
            return;
        }

        for (Auditory a: auditories ) {
            if (a.getId() == auditory.getId()) {
                throw new IllegalArgumentException("This auditory already
assigned to this employee.");
            }
        }

        int n = auditories.length;

        Auditory[] newAuditories = new Auditory[n + 1];

        for (int i = 0; i < n; i++) {
            newAuditories[i] = auditories[i];
```

```
        }

        newAuditories[n] = auditory;
        auditories = newAuditories;
    }

    public void sortAuditoriesById() {
        if (auditories == null) {
            return;
        }

        int n = auditories.length;
        Auditory[] sortedAuditories = auditories.clone();

        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (sortedAuditories[j].getId() > sortedAuditories[j +
1].getId()) {
                    Auditory temp = sortedAuditories[j];
                    sortedAuditories[j] = sortedAuditories[j + 1];
                    sortedAuditories[j + 1] = temp;
                }
            }
        }

        auditories = sortedAuditories;
    }

    public void giveRaise(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("Raise amount cannot be
negative.");
        }
        salary += amount;
    }

    public void giveRaise(double percentage, boolean isPercentage) {
        if (isPercentage) {
            if (percentage < 0) {
                throw new IllegalArgumentException("Raise percentage cannot
be negative.");
            }

            double amount = salary * (percentage / 100);
            salary += amount;
        }
    }
}
```

programme/employee/Laborant.java:

```
package employee;

import auditory.Auditory;

public class Laborant extends Employee {
    private int masterKeyId;

    public Laborant(String name, double salary, Auditory[] auditories, int
masterKeyId) {
        super(name, salary, auditories);
        checkMasterKeyId(masterKeyId);
        this.masterKeyId = masterKeyId;
    }

    public int getMasterKeyId() {
        return masterKeyId;
    }

    public void setMasterKeyId(int masterKeyId) {
        checkMasterKeyId(masterKeyId);
        this.masterKeyId = masterKeyId;
    }

    private void checkMasterKeyId(int masterKeyId) {
        Auditory[] auditories = this.getAuditories();
        boolean hasMasterAuditory = false;

        if (masterKeyId <= 0) {
            throw new IllegalArgumentException("Master key ID must be a
positive integer.");
        }

        if (auditories != null) {
            for (Auditory a : auditories) {
                if (a.getId() == masterKeyId) {
                    hasMasterAuditory = true;
                    break;
                }
            }
        }

        if (!hasMasterAuditory) {
            throw new IllegalArgumentException("Master key ID must correspond
to one of the assigned auditories.");
        }
    }

    @Override
    public String getEmployeeData() {
```

```
        return super.getEmployeeData() + ", Role: Laborant" + (masterKeyId >
-1 ? ", Master Key: " + masterKeyId : "");
    }
}
```

programme/Main.java:

```
import java.util.Random;

import auditory.Auditory;
import employee.Employee;
import employee.Laborant;

public class Main {
    static Random random = new Random();

    private static void printEmployees(Employee[] employees) {
        for (int i=0; i<employees.length; i++) {
            System.out.println(i + 1 + ". " +
employees[i].getEmployeeData());
        }
    }

    private static void getAuditoriesForEmployee(Employee employee) {
        Auditory[] auditories = employee.getAuditories();
        System.out.println(employee.getName() + " works in:");

        if (auditories == null || auditories.length == 0) {
            System.out.println("\tNo auditories assigned");
            return;
        }

        for (Auditory a: auditories) {
            System.out.println("\t" + "Auditory " + a.getId() + " (" +
a.getForSubject() + ")");
        }
    }

    public static void main(String[] args) {
        Auditory a1 = new Auditory(1, "Informatics", 70);
        Auditory a2 = new Auditory(2, "Mathematics", 30);
        Auditory a3 = new Auditory(1, "Physics", 50);

        Auditory.Equipment projector = a1.new Equipment("Projector", true);
        System.out.println("--- EQUIPMENT TEST ---");
        System.out.println(projector.getStatus());

        Employee[] employees = new Employee[5];

        employees[0] = new Employee();
        employees[1] = new Employee("Jane Smith", 6000, new Auditory[]{a2});
```

```
        employees[2] = new Laborant("Alice Johnson", 7000, new Auditory[]{a2,
a3}, 2);
        employees[3] = new Laborant("Bob Brown", 5500, new Auditory[]{a1},
1);
        employees[4] = new Laborant("Charlie Davis", 6500, new
Auditory[]{a2}, 2);

        class EmployeesAnalyzer {
            public double getAverageSalary(Employee[] employees) {
                double totalSalary = 0;
                int emptyEmployeesCount = 0;

                for (Employee employee : employees) {
                    if (employee.getSalary() == 0) {
                        emptyEmployeesCount++;
                    }
                    totalSalary += employee.getSalary();
                }

                return totalSalary / (Employee.getTotalEmployees() -
emptyEmployeesCount);
            }

            public Employee getHighestSalaryEmployee(Employee[] employees) {
                Employee topEmployee = employees[0];

                for (Employee employee : employees) {
                    if (employee.getSalary() > topEmployee.getSalary()) {
                        topEmployee = employee;
                    }
                }
                return topEmployee;
            }
        }

        EmployeesAnalyzer analyzer = new EmployeesAnalyzer();

        System.out.println("\n--- EMPLOYEE LIST ---");
        printEmployees(employees);

        System.out.println("\n--- ASSIGNMENT CHECK ---");
        for (Employee e: employees) {
            getAuditoriesForEmployee(e);
        }

        System.out.println("\n--- STATISTICS ---");
        Employee highestSalaryEmployee =
analyzer.getHighestSalaryEmployee(employees);
```

```
        System.out.println("Highest salary employee: " +
highestSalaryEmployee.getName() + " - $" +
highestSalaryEmployee.getSalary());
        System.out.println("Average salary value: $" +
analyzer.getAverageSalary(employees));
    }
}
```

Результати тестування класу/Результати дослідження відношення спадкування:

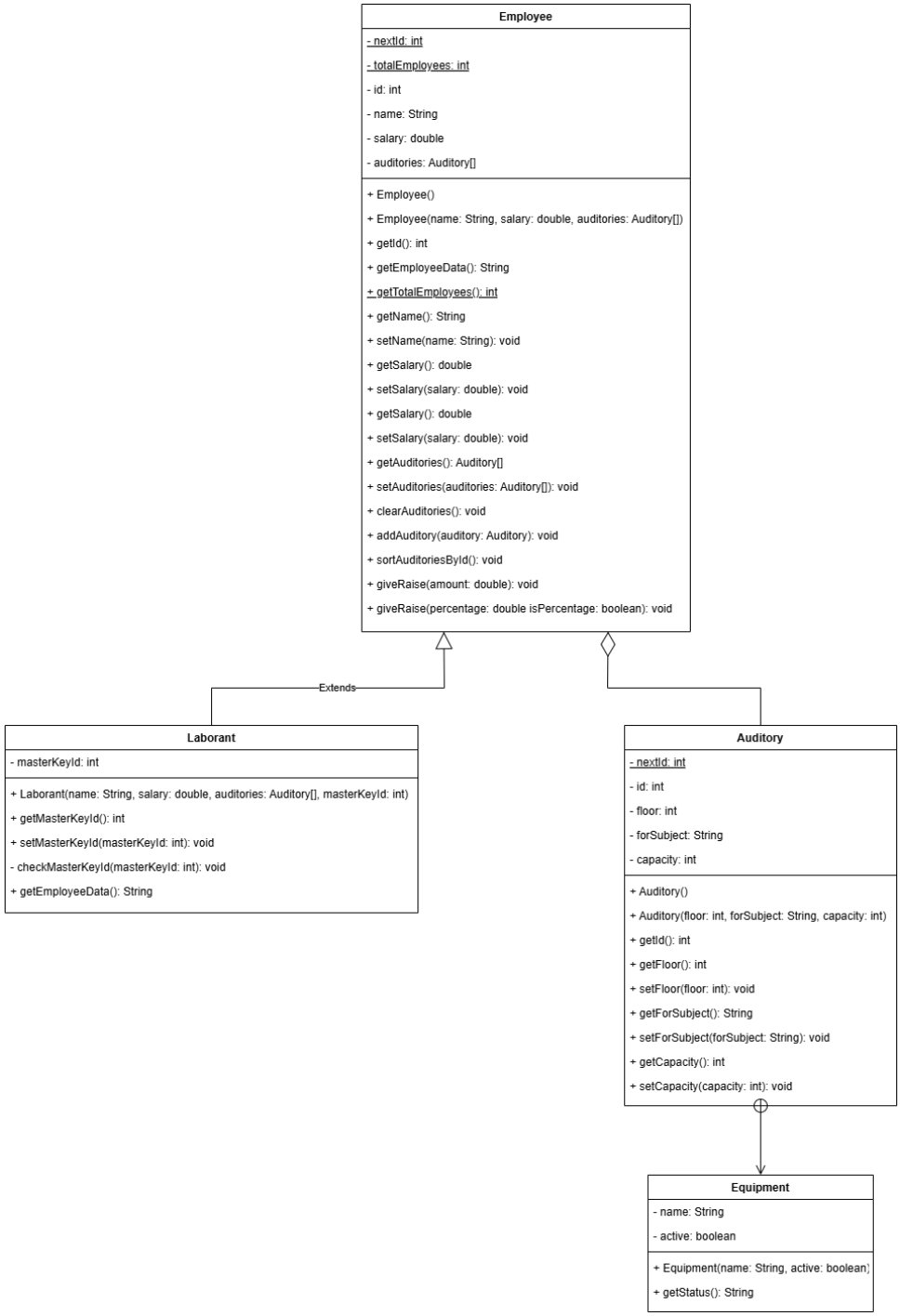
```
--- EQUIPMENT TEST ---
Projector is active in auditory 1

--- EMPLOYEE LIST ---
1. ID: 1, Name: Unknown, Salary: $0.0
2. ID: 2, Name: Jane Smith, Salary: $6000.0
3. ID: 3, Name: Alice Johnson, Salary: $7000.0, Role: Laborant, Master Key: 2
4. ID: 4, Name: Bob Brown, Salary: $5500.0, Role: Laborant, Master Key: 1
5. ID: 5, Name: Charlie Davis, Salary: $6500.0, Role: Laborant, Master Key: 2

--- ASSIGNMENT CHECK ---
Unknown works in:
        No auditories assigned
Jane Smith works in:
        Auditory 2 (Mathematics)
Alice Johnson works in:
        Auditory 2 (Mathematics)
        Auditory 3 (Physics)
Bob Brown works in:
        Auditory 1 (Informatics)
Charlie Davis works in:
        Auditory 2 (Mathematics)

--- STATISTICS ---
Highest salary employee: Alice Johnson - $7000.0
Average salary value: $6250.0
PS C:\archive\EducationalWork\KPI\Java\КП2\programme>
```

Діаграма класів:



Висновок

Під час виконання комп’ютерного практикуму досліджено механізми відношень між класами в Java, а саме спадкування, агрегування, вкладені та локальні класи, на прикладі розробки класів «співробітник», «лаборант» та «аудиторія».

Головною перевагою спадкування є можливість повторного використання коду: клас-нащадок (лаборант) автоматично отримує функціонал базового класу

(співробітник), що усуває дублювання коду. Це також дозволяє застосовувати поліморфізм для зручної обробки різних об'єктів через спільний масив.

Основним недоліком спадкування є сильна зв'язність класів: зміни в базовому класі можуть непередбачувано порушити роботу нащадків. Крім того, нащадок іноді успадковує непотрібний йому функціонал, а жорстка ієрархія ускладнює перевикористання цих класів у відокремлених сторонніх проектах.

Контрольні запитання

1. ***За яких умов класи знаходяться у відношенні агрегування? Композиції? Спадкування? Вкладення?***

    - Агрегування має місце, коли клас *A* містить поля, які є екземплярами класу *B*.
    - Композиція має місце, якщо екзмепляри класу *B* не мають самостійного призначення, а виступають виключно як компоненти класу *A*.
    - Спадкування виникає, коли клас-нащадок будується на основі вже існуючого батьківського класу, переймаючи всі його властивості та методи.
    - Вкладення має місце, коли один клас оголошується безпосередньо всередині тіла іншого класу.

2. ***Коли доцільно використовувати агрегування/спадкування/вкладення класів?***

    - Агрегування доцільно, коли об’єкти логічно складаються з інших об’єктів, але ці компоненти можуть існувати незалежно і змінюватися під час виконання програми.
    - Спадкування доцільно використовувати для опису загальних властивостей множини класів, щоб уникнути дублювання коду, а також, щоб обробляти ріщні об’єкти в одному масиві суперкласу за допомогою поліморфізму.
    - Вкладення доцільно, коли допоміжний клас тісно пов’язаний з основним, використовується виключно ним і не повинен бути видимим або доступним ззовні без прив’язки до об’єкта-власника.

3. ***Що представляє діаграма класів?***

   Діаграма класів представляє ієрархію класів та зв'язки між ними. Вона показує структуру ОО програми.

4. ***Як розробити метод доступу до поля/методу, яке належить до іншого класу? При яких парах модифікаторів доступу це можливе?***

   Щоб отримати доступ до приватного поля іншого класу, потрібно в цьому класі розробити публічний метод доступу (н.п. *getField()*). Елемети класу можуть позначатися як *public* – доступні ззовні класу, *private* – доступні лише всередині класу, *protected* – доступні всередині класу та для класів-нащадків.

5. ***Які особливості розробки масивів мовою програмування Java?***

   Масиви в Java є посилальними типами (об'єктами). Головна особливість в ООП – це те, що тип масиву визначає множину типів, які він може містити. Таким чином об'єкт батьківського класу може містити об'єкти будь-яких своїх класів-нащадків.

6. ***Як написати фрагмент Java-коду, що відшукує мінімальне значення масиву цілих значень?***

```
int min = array[0]
for (int i = 1; i < array.length; i++) {
    if (array[i] < min) {
        min = array[i];
    }
}
return min;
```

7. ***Як написати рядок Java-коду, що надає доступ до методу getC() поля класу A, якщо поле c є екземпляром класу B? При якому відношенні між класами A і B це можливо?***

```
a.getC().getC()
```

   Це можливо при відношенні агрегування або композиції, коли клас *A* містить поле, яке є екземпляром класу *B*.

8. ***У якому відношенні можуть знаходитись класи «студент»-«група», «розклад»-«пара», «світлофор»-«перехрестя», «дуга»-«граф»?***

- «студент»-«група» - агрегування (студент має відповідну групу).
- «розклад»-«пара» - агрегування або кмпозиція.
- «світлофор»-«перехрестя» - агрегування (перехрестя має свіфтлофори).
- «дуга»-«граф» - композиція (дуга не має сенсу без графу).

9. ***Чи можуть знаходитись у відношенні агрегунвання/спадкування класи «кімната»-«будівля», «студент»-«супермаркет», «авто»-«населений пункт», «супермаркет»-«покупець»?***

- «кімната»-«будівля»: агрегування (будівля має кімнати); спадкування неможливе.
- «студент»-«супермаркет» - ні агрегування, ні спадкування.
- «авто»-«населений пункт» - агрегування (авто перебуває в населеному пункті); спадкування неможливе.
- «супермаркет»-«покупець» - агрегування можливе, але залежить від контексту (н.п. покупець має обраний супермаркет в застосунку).